

Festival Planner

Datei- und Verantwortlichkeitsübersicht

Version v0.2 · Stand 2026-09-17 · Open Lab Session „Vom Vibe Coding zum Software Engineering“

Diese Übersicht beschreibt für jede Datei im Projekt `festival-planner`, wofür sie zuständig ist. Für die Markdown-Dokumentation (`*.md`) gibt es zusätzlich eine Spalte „Wann ändern?“ – als Orientierung, wie oft und aus welchem Anlass eine Datei typischerweise angepasst wird.

1. Projektstruktur im Überblick

```
festival-planner/
├── CLAUDE.md           Arbeitsregeln & Kontext für Claude Code
├── README.md          Einstieg für Workshop-Teilnehmer
├── requirements.txt    Laufzeit-Abhängigkeiten
├── requirements-dev.txt + Test-Abhängigkeiten (pytest, httpx)
├── .gitignore         von Git ausgeschlossene Dateien/Ordner
├── festival.db        (generiert, nicht in Git)
├──
├── app/              Anwendungscode (Backend)
│   ├── __init__.py   Composition Root (FastAPI-App)
│   ├── main.py       API-Endpunkte
│   ├── routers.py    Datenbank-Queries
│   ├── crud.py       ORM-Modelle
│   ├── models.py     Business-Logik
│   ├── schedule.py   Datenbank-Infrastruktur
│   ├── db.py         Seed-Skript
│   └── seed.py
├──
├── static/           Frontend (HTML/CSS/Vanilla JS)
│   ├── index.html
│   ├── style.css
│   └── app.js
├──
├── tests/            Automatisierte Tests
│   ├── test_schedule.py
│   └── test_api.py
├──
├── doc/              Projektdokumentation
│   ├── roadmap.md
│   ├── requirements.md
│   ├── requirements-history/
│   │   └── requirements-v0.1-mvp.md (eingefrorener Snapshot)
│   ├── domain-model.md
│   ├── architecture.md
│   ├── backlog.md
│   ├── review.md
│   └── claude-install.md
├──
└── teaching/         Material nur für Teilnehmer (write-only für Claude)
    └── ai-prompts.md
```

2. Anwendungscode – `app/`

Datei	Verantwortung	Abhängig von
<code>__init__.py</code>	Leer – macht <code>app/</code> zu einem importierbaren Python-Package.	–
<code>main.py</code>	Composition Root: erzeugt das FastAPI-App-Objekt, startet <code>init_db()</code> über den Lifespan, bindet <code>routers.py</code> und <code>static/</code> ein. Enthält selbst	<code>db</code> , <code>routers</code>

Datei	Verantwortung	Abhängig von
	keine Endpunkte.	
<code>routes.py</code>	Die zwei HTTP-Endpunkte <code>GET /api/program</code> und <code>GET /api/stages</code> sowie die Pydantic-Antwortmodelle. Ruft <code>crud.py</code> für Daten und <code>schedule.py</code> für den Status ab.	<code>crud</code> , <code>schedule</code> , FastAPI
<code>crud.py</code>	Datenbank-Queries als einfache Funktionen: Bühnenliste (sortiert), Programmliste (Join <code>Act / Artist / Stage</code> , sortiert, optional gefiltert). Keine Repository-Klassen.	<code>models</code> , <code>db</code> (Session)
<code>models.py</code>	ORM-Modelle <code>Artist</code> , <code>Stage</code> , <code>Act</code> inkl. Beziehungen (<code>relationship()</code>), siehe <code>doc/domain-model.md</code> .	SQLAlchemy, <code>db.Base</code>
<code>schedule.py</code>	Reine Business-Logik ohne DB/HTTP: <code>FESTIVAL_TZ</code> (UTC+02:00), <code>festival_now()</code> , Statusregeln „läuft jetzt“ / „als Nächstes“. Leicht unit-testbar, da <code>now</code> als Parameter übergeben wird.	nur <code>datetime</code>
<code>db.py</code>	Reine Infrastruktur: SQLite-Engine, Session-Factory, <code>Base</code> , <code>init_db()</code> , FastAPI-Dependency <code>get_db()</code> . Kein Modell mehr enthalten.	SQLAlchemy
<code>seed.py</code>	Skript <code>python -m app.seed</code> : legt Tabellen an, löscht vorhandene Zeilen, erzeugt Artists/Stages und ca. 10–15 Acts auf 3 Bühnen für das heutige Datum.	<code>db</code> , <code>models</code> , <code>schedule</code>

3. Frontend – `static/`

Datei	Verantwortung
<code>index.html</code>	Die einzige Seite der Anwendung – Grundgerüst für Liste, Bühnenfilter und Statusanzeige.
<code>style.css</code>	Layout (mobiltauglich ab 360 px Breite), visuelle Hervorhebung der Status „now“/„next“.
<code>app.js</code>	Vanilla JS ohne Build-Tooling: ruft <code>/api/stages</code> und <code>/api/program</code> ab, rendert die Liste, reagiert auf den Bühnenfilter.

4. Tests – `tests/`

Datei	Verantwortung
<code>test_schedule.py</code>	Unit-Tests der Business-Logik in <code>schedule.py</code> – ohne DB, ohne HTTP. Schwerpunkt: Randfälle bei „now“/„next“ (exakte Start-/Endzeit, parallele Acts, gleiche Startzeiten, gefilterte Liste, leere Liste).
<code>test_api.py</code>	API-Tests mit FastAPIs <code>TestClient</code> gegen eine In-Memory-SQLite-Datenbank (<code>StaticPool</code>). Ersetzt <code>get_db</code> und <code>festival_now</code> per <code>dependency_overrides</code> . Prüft Sortierung, Bühnenfilter, Statusfeld, leere Datenbank.

5. Markdown-Dokumentation – Aufgabe & „Wann ändern?“

Diese Tabelle ist der eigentliche Kern der Übersicht: Sie zeigt, welche Aufgabe jede *.md -Datei im Projekt hat und – wichtiger für den Alltag – **wie oft und aus welchem Anlass** sie normalerweise geändert wird. Das hilft zu entscheiden, in welche Datei eine neue Information gehört, statt alles in eine einzige Datei zu schreiben.

Datei	Aufgabe	Wann ändern?
CLAUDE.md	Zentraler Projektkontext für Claude Code: Projektziel, Tech-Stack, Projekt-Entscheidungen, Kurzfassungen von Architektur/Domain Model, Development Rules, Projekt-Commands, Requirements Policy.	Selten – nur wenn sich grundlegende Regeln, Entscheidungen oder Befehle ändern. Wird nicht bei jeder Story angepasst.
README.md	Einstiegspunkt für Workshop-Teilnehmer: Projektziel, Setup (venv, Dependencies), wie man Claude Code startet.	Selten – nur bei geändertem Setup-Ablauf oder neuen Voraussetzungen.
doc/claude-install.md	Schritt-für-Schritt-Anleitung zur Installation von Claude Code unter VS Code.	Nur bei geänderter Installationsanleitung (z. B. neue Extension-Version, neuer Ablauf).
doc/roadmap.md	Phasen und Vorgehen des Gesamtprojekts (Schritt 1–22), inkl. „Phase 2 – Refactoring v0.2“.	Bei neuen Phasen/Meilensteinen oder wenn sich die grobe Reihenfolge des Vorgehens ändert.
doc/requirements.md	Aktuell gültige fachliche Anforderungen (Muss/optional), Benutzeraktionen, Scope-Abgrenzung.	Nur bei echten Anforderungsänderungen. Der jeweils überholte Stand wird nicht überschrieben, sondern als Snapshot eingefroren (siehe unten).
doc/requirements-history/requirements-v0.1-mvp.md	Eingefrorener Snapshot der ursprünglich bestätigten v0.1-MVP-Anforderungen, vor dem Domain-Modell-Refactor.	Nie – historische Referenz. Neue eingefrorene Stände kommen als weitere Datei dazu, bestehende werden nicht mehr geändert.
doc/domain-model.md	Fachliches Datenmodell: Entitäten (Artist , Stage , Act), Attribute, Beziehungen, Invarianten, ER-Diagramm.	Nur bei Modelländerungen (neue Entität, neues Attribut, geänderte Beziehung/Invariante).

Datei	Aufgabe	Wann ändern?
<code>doc/architecture.md</code>	Architekturentscheidungen, Projektstruktur, Verantwortlichkeiten je Datei, HTTP-API-Vertrag, Datenfluss, bewusste Nicht-Entscheidungen und Erweiterungspunkte.	Nur bei Architekturänderungen (neue Datei/Schicht, geänderter API-Vertrag, geänderte Leitlinie).
<code>doc/backlog.md</code>	User Stories und technische Aufgaben mit Priorität, Abhängigkeiten und Status (offen/in Arbeit/erledigt).	Laufend – nach jeder abgeschlossenen Story/Aufgabe und bei neuer Priorisierung. Häufigste Änderungsfrequenz aller Dokumente.
<code>doc/review.md</code>	Ergebnisse unabhängiger Code-Reviews: erfüllte Akzeptanzkriterien, Findings, offene Änderungswünsche, Gesamturteil.	Bei Reviews – nicht bei jedem Commit. Neue Reviews werden als weiterer Abschnitt/Nachtrag ergänzt, nicht als Ersatz des vorherigen.
<code>teaching/ai-prompts.md</code>	Schulungsmaterial nur für Teilnehmer. Für Claude Code laut <code>CLAUDE.md</code> „write only“ – wird hier bewusst nicht inhaltlich beschrieben.	Durch die Workshop-Leitung nach Bedarf; nicht Teil des regulären Projekt-Dokumentationszyklus.

Hinweis zu `backlog.md` : In größeren, länger laufenden Projekten wird ein Backlog oft aus dem Repository ausgelagert (z. B. GitHub Projects/Issues), damit Status und Zuweisungen nicht manuell in Markdown gepflegt werden müssen. Für dieses Schulungsprojekt bleibt `backlog.md` bewusst als einfache Markdown-Datei im Repo – konsistent mit der Leitlinie „keine unnötigen zusätzlichen Werkzeuge“.

6. Sonstige Dateien im Wurzelverzeichnis

Datei	Verantwortung
<code>requirements.txt</code>	Laufzeit-Abhängigkeiten: <code>fastapi</code> , <code>uvicorn[standard]</code> , <code>sqlalchemy</code> .
<code>requirements-dev.txt</code>	<code>-r requirements.txt</code> plus <code>pytest</code> und <code>httpx</code> – ausschließlich für die Entwicklung, nicht für den Betrieb.
<code>.gitignore</code>	Schließt generierte/lokale Dateien von Git aus: <code>__pycache__/</code> , <code>.venv/</code> , <code>*.db</code> , IDE- und OS-Dateien, <code>.pytest_cache/</code> .
<code>festival.db</code>	Generierte SQLite-Datenbank (durch <code>python -m app.seed</code> bzw. beim App-Start). Nicht in Git, jederzeit neu erzeugbar.

Datei	Verantwortung
<code>.pytest_cache/</code>	Von pytest automatisch erzeugter Cache. Kein Projektinhalt, nicht in Git.